ZentroMart Voice Support Agent

VoiceOps 2026 Capstone Report | 24 October 2026

Abstract. This project builds a voice-enabled customer-support agent for ZentroMart, a fictional Indian marketplace for groceries and electronics. The agent answers from the supplied policy documents, verifies a caller before looking up an order, carries context across a conversation, and escalates cases that need a human decision. Gnani provides both speech recognition and speech synthesis; the support logic is shared by chat and voice.

The problem

The support team repeatedly receives questions about late deliveries, cancellations, return eligibility, refunds, damaged items, and missing parcels. A useful assistant must be helpful without becoming overconfident: it cannot invent a policy, disclose an order from an ID alone, or claim that a ticket or refund has been created when it has not. The project was designed around those constraints rather than around open-ended conversation alone.

What I built

The application combines a FastAPI service, a browser chat and voice interface, a Markdown retrieval layer, CSV-backed order tools, and a Gnani speech adapter. Policy files are split at their supplied section IDs. Each chunk retains its source document, policy status, effective date, and document type. Retrieval favours current policies over agent guides, help pages, FAQs, and marketing material; the superseded return policy is excluded.

For order-specific requests, the assistant asks for an 8-digit order ID and then either the last four digits of the registered phone number or the delivery pincode. Only a successful call to the supplied lookup tool unlocks order status, dates, return, or refund records. Verification values are not repeated in replies and are redacted from handover summaries.

Conversation design

The agent keeps short-lived conversation state for the active order, verification result, pending request, return context, and ticket state. It accepts digit-by-digit order IDs and pincodes from speech, handles corrections such as “sorry, I made a mistake,” and answers follow-up questions without asking the caller to repeat a verified order ID. It can answer a combined request, for example delivery timing plus the refund process for a future return, in one response with the relevant policy evidence.

When a customer asks for a human, reports a safety or fraud concern, threatens legal action, requests an exception, or repeatedly contacts support about an unresolved issue, the agent creates a handoff ticket through the supplied tool. It confirms success only after the tool reports *created*=*True*. If no order ID is available, the customer is told plainly that the ticket is general and is asked for the ID to help the reviewer.

Evaluation and findings

The supplied dataset validator passed for the complete synthetic pack: 9 CSV tables, 21 knowledge-base documents, 1,000 orders, 1,783 order items, 52 single questions, and 23 conversations. The local evaluation replays the practice data and records the expected and actual decision, evidence, tool calls, and any failure reason. Ticket creation is mocked during replay so evaluation never writes live tickets.

Evaluation area	Result	Interpretation
Step 1: prompt-only	Not measured	A comparable hosted-model baseline was not run; this report does not invent one.
Step 2: single questions	52 / 52 (100%)	Strict deterministic pass rate across decision, facts, and evidence checks.
Step 2: evidence	47 / 47 cases	All listed gold evidence retrieved in every applicable case.
Step 3: conversations	96 / 96 (100%)	Turn-level result across the 23 supplied conversations.
Step 3: tool calls	54 / 54	Expected tool type and order ID matched whenever specified.
Regression suite	169 passed; 86% coverage	Coverage across the src package.

These results are useful because they are reproducible against the supplied dataset, but they are not a hosted-LLM or production-accuracy claim. The next iteration should run a genuine prompt-only baseline against the same 52 questions to provide the missing Step 1 comparison.

Voice experience

The voice layer uses Gnani only. Browser WebM recordings are converted locally to 16 kHz WAV because the Gnani STT interface accepts AAC, FLAC, M4A, MP3, OGG, and WAV. The interface validates size and duration, returns a retry prompt for an empty transcript, and exposes service errors instead of pretending a reply was generated. Replies are deliberately brief so they sound natural when spoken.

The interface supports STT in Bengali, English, Gujarati, Hindi, Kannada, Malayalam, Marathi, Punjabi, Tamil, and Telugu. The en-IN mode supports English mixed with Hindi and returns Latin script. TTS has the same choices plus Hinglish (hi-en), and the listening and reply languages can be selected independently. Kaveri, Trupti, Pranav, and Shlok are available voices.

Reproducible submission and next steps

Required batch command

```
python run.py path\to\audio-folder --output sample_outputs --language en-IN
--reply-language hi-en --voice Kaveri
```

The command accepts a folder of customer audio files and processes each independently. For every input it produces a spoken WAV reply and a JSON result containing the input filename, transcript, text reply, reply-audio filename, tool calls, source section IDs, and the full retrieved document chunks. The chunk record includes the section ID, title, source document, and exact text used as evidence. This gives the organizers both the spoken output and a traceable grounding record.

What did not work perfectly

The project is a working prototype, not a finished production agent. The Step 1 score remains unavailable until a hosted prompt-only baseline is run. A live Gnani test with a self-recorded clip is still needed for each language/voice combination, including background-noise and playback checks. These limitations are stated directly because a reliable submission should distinguish verified behavior from work that remains.

What I would do next

First, run a prompt-only baseline and compare it against the RAG configuration using the same evaluator. Next, record a compact two-minute demo that shows an order lookup, a follow-up question, and one human handoff. Finally, publish the repository with README.md, .env.example, requirements.txt, run.py, src/, report.pdf, demo.mp4, and sample_outputs/; exclude .env, API keys, virtual environments, and a duplicate dataset copy.

Conclusion

ZentroMart demonstrates the intended VoiceOps pipeline: customer audio to Gnani STT, a grounded and tool-aware support agent, then Gnani TTS. Its strongest features are verification-gated order handling, traceable policy evidence, multilingual voice configuration, and conversation continuity. The offline practice evaluation passes all supplied cases; the remaining work is the hosted baseline, live voice validation, and public demo/submission links.